

Lecture 10

Finite Element Method: general weak formulation for non-symmetric operators, Galerkin's method, element matrices, 1D FEM system matrix assembly, making FEM solver in Python and FEniCSx, 2D FEM system matrix assembly on non-uniform meshes

Non-symmetric operators: general Finite Element Method

Non-symmetric operators

Consider the convection-diffusion equation:

$$\begin{aligned} -\nabla \cdot (k \nabla u) + \rho \mathbf{v} \cdot \nabla u + cu &= f, \quad \mathbf{x} \in \Omega \\ u(\mathbf{x}) &= g_D(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_D \\ -k(\mathbf{x}) \nabla u(\mathbf{x}) \cdot \mathbf{n} &= g_N(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_N \\ \sigma u(\mathbf{x}) - k(\mathbf{x}) \nabla u(\mathbf{x}) \cdot \mathbf{n} &= g_R(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_R \end{aligned}$$

- The differential operator is no longer self-adjoint on $U(\Omega)$
- There is no suitable functional $F(u)$ that one could minimize on $U(\Omega)$

General weak formulation

- Introduce two function spaces:
 - (1) *Solution space* $U(\Omega) = \{\text{'smooth' } u \mid u(\mathbf{x}) = g_D(\mathbf{x}), \mathbf{x} \in \partial\Omega_D\}$
 - (2) *Test space* $V(\Omega) = \{\text{'smooth' } v \mid v(\mathbf{x}) = 0, \mathbf{x} \in \partial\Omega_D\}$,
also called space of *test functions*
- Find $u \in U(\Omega)$ such that

$$\int_{\Omega} [-\nabla \cdot (k \nabla u) + \rho \mathbf{v} \cdot \nabla u + cu - f] v \, d\Omega = 0$$

for all $v \in V(\Omega)$

- Such u is then called a *weak solution* of the original problem. It solves the problem in a weak sense, because integration allows for weaker smoothness requirements on u and v

Reducing the order of derivatives

To introduce Neumann and Robin boundary conditions, reduce the order of the highest partial derivative:

$$\begin{aligned} & \int_{\Omega} [-\nabla \cdot (k \nabla u) + \rho \mathbf{v} \cdot \nabla u + cu - f] v \, d\Omega \\ &= \int_{\Omega} [-\nabla \cdot (k \nabla u)] v \, d\Omega + \int_{\Omega} [\rho \mathbf{v} \cdot \nabla u + cu - f] v \, d\Omega \\ &= \int_{\Omega} k(\nabla u \cdot \nabla v) \, d\Omega - \int_{\partial\Omega} k(\nabla u \cdot \mathbf{n}) v \, d\Gamma + \int_{\Omega} [\rho \mathbf{v} \cdot \nabla u + cu - f] v \, d\Omega \end{aligned}$$

Now, also 'smooth' can be specified as:

$$u \in U(\Omega) = \{u \in H^1(\Omega) \mid u(\mathbf{x}) = g_D(\mathbf{x}), \mathbf{x} \in \partial\Omega_D\}$$

$$v \in V(\Omega) = \{v \in H^1(\Omega) \mid v(\mathbf{x}) = 0, \mathbf{x} \in \partial\Omega_D\}$$

$H^1(\Omega)$ is indeed a weaker smoothness requirement than $C^2(\Omega)$

Standard weak formulation

Apply the *natural* BC's in the boundary integral (over $\partial\Omega$)

$$\begin{aligned}-k(\mathbf{x})\nabla u(\mathbf{x}) \cdot \mathbf{n} &= g_N(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_N \\ \sigma u(\mathbf{x}) - k(\mathbf{x})\nabla u(\mathbf{x}) \cdot \mathbf{n} &= g_R(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega_R\end{aligned}$$

The Dirichlet part of the boundary integral vanishes since $v \in V(\Omega)$.

The standard weak formulation of the problem: find $u \in U(\Omega) = \{u \in H^1(\Omega) \mid u(\mathbf{x}) = g_D(\mathbf{x}), \mathbf{x} \in \partial\Omega_D\}$ such that

$$\begin{aligned}& \int_{\Omega} [k(\nabla u \cdot \nabla v) + \rho(\mathbf{v} \cdot \nabla u)v + cuv] d\Omega + \int_{\partial\Omega_R} \sigma uv d\Gamma \\ &= \int_{\Omega} f v d\Omega - \int_{\partial\Omega_N} g_N v d\Gamma - \int_{\partial\Omega_R} g_R v d\Gamma\end{aligned}$$

for all $v \in V(\Omega) = \{v \in H^1(\Omega) \mid v(\mathbf{x}) = 0, \mathbf{x} \in \partial\Omega_D\}$

Galerkin and Petrov-Galerkin methods

- Introduce the finite-dimensional *approximation space* $U_h(\Omega)$ with the basis functions $\{u_i(\mathbf{x})\}$, $i = 0, \dots, n$; so that any $u_h \in U_h(\Omega)$ can be represented as

$$u_h(\mathbf{x}) = \sum_{i=0}^n a_i u_i(\mathbf{x})$$

The essential Dirichlet BC's must be included in $U_h(\Omega)$ on the level of some 'essential' basis functions

- **Galerkin Mehtod:** As a basis for $V_h(\Omega)$ use the basis functions $\{u_i(\mathbf{x})\}$ from $U_h(\Omega)$ with $i \in S_{NE}$. The set of 'non-essential' indexes S_{NE} excludes those basis functions that were used to enforce the essential Dirichlet BC's on $U_h(\Omega)$
- **Petrov-Galerkin Mehtod:** Use a completely different $V_h(\Omega)$ described by some other functions $\{v_i(\mathbf{x})\}$, $i \in S$, where $\dim(V_h) = \dim(S_{NE})$

Galerkin's method

- Find coefficients $\{a_i\}$, $i \in S_{NE}$ by solving the following system of linear algebraic equations:

$$\sum_{i=0}^n a_i \left\{ \int_{\Omega} [k(\nabla u_i \cdot \nabla u_j) + \rho(\mathbf{v} \cdot \nabla u_i)u_j + cu_i u_j] d\Omega + \int_{\partial\Omega_R} \sigma u_i u_j d\Gamma \right\} \\ = \int_{\Omega} f u_j d\Omega - \int_{\partial\Omega_N} g_N u_j d\Gamma - \int_{\partial\Omega_R} g_R u_j d\Gamma, \quad j \in S_{NE}$$

The coefficients a_i corresponding to 'essential' indexes are fixed by the Dirichlet BC's as in the Ritz's method by requiring that

$$u_h(\mathbf{x}) = \sum_{i=0}^n a_i u_i(\mathbf{x}) = g_D(\mathbf{x})$$

and noticing that most of the $u_i(\mathbf{x}) = 0$, for $\mathbf{x} \in \partial\Omega_D$.

Example: Galerkin method in 1D

Apply Galerkin FEM to discretize the convection-diffusion equation:

$$-ku'' + ru' + cu = f, \quad \mathbf{x} \in \Omega = (0, 1)$$

$$u(0) = a, \quad -ku'(1) = b$$

$$k, r, c \text{ are constants}$$

Step 1: weak formulation

$$-ku'' + ru' + cu = f,$$

$$-ku'' + ru' + cu - f = 0,$$

$$(-ku'' + ru' + cu - f)v = 0,$$

$$\int_0^1 (-ku'' + ru' + cu - f)v \, dx = 0,$$

$$-k \int_0^1 u''v \, dx + r \int_0^1 u'v \, dx + c \int_0^1 uv \, dx - \int_0^1 fv \, dx = 0,$$

$$-k \left[u'v \Big|_0^1 - \int_0^1 u'v' \, dx \right] + r \int_0^1 u'v \, dx + c \int_0^1 uv \, dx - \int_0^1 fv \, dx = 0,$$

$$\begin{aligned} & -ku'(1)v(1) + ku'(0)v(0) + k \int_0^1 u'v' \, dx \\ & + r \int_0^1 u'v \, dx + c \int_0^1 uv \, dx - \int_0^1 fv \, dx = 0. \end{aligned}$$

Step 2: function spaces and boundary conditions

Domain: $\Omega = (0, 1)$.

Solution space: $u \in U(\Omega) = \{u \in H^1(\Omega) \mid u(0) = a\}$.

Test space: $v \in V(\omega) = \{v \in H^1(\Omega) \mid v(0) = 0\}$.

Neumann BC: $-ku'(1) = b$.

$$\begin{aligned} & -ku'(1)v(1) + ku'(0)v(0) + k \int_0^1 u'v' \, dx \\ & + r \int_0^1 u'v \, dx + c \int_0^1 uv \, dx - \int_0^1 fv \, dx = 0, \\ & bv(1) + k \int_0^1 u'v' \, dx + r \int_0^1 u'v \, dx + c \int_0^1 uv \, dx - \int_0^1 fv \, dx = 0, \\ & k \int_0^1 u'v' \, dx + r \int_0^1 u'v \, dx + c \int_0^1 uv \, dx = \int_0^1 fv \, dx - bv(1). \end{aligned}$$

Step 3: subspaces and Galerkin equations

Let $u_i(x)$, $i = 0, \dots, n$ be the basis functions.

Solution subspace: $U_h(\Omega) = \{u = \sum_{i=0}^n a_i u_i \mid u(0) = a\}$.

Test subspace: $V_h(\Omega) = \{v = \sum_{i=0}^n b_i u_i \mid v(0) = 0\}$.

Substitute: $u = \sum_{i=0}^n a_i u_i$.

Choose: $v = u_j$, where $j \in S_{NE}$ (i.e. excluding basis functions u_j that are nonzero at the Dirichlet boundary $x = 0$).

$$k \int_0^1 u' v' dx + r \int_0^1 u' v dx + c \int_0^1 uv dx = \int_0^1 f v dx - b v(1),$$

$$k \int_0^1 \sum_{i=0}^n a_i u_i' u_j' dx + r \int_0^1 \sum_{i=0}^n a_i u_i' u_j dx + c \int_0^1 \sum_{i=0}^n a_i u_i u_j dx$$

$$= \int_0^1 f u_j dx - b u_j(1), \quad j \in S_{NE};$$

$$\sum_{i=0}^n \left[k \int_0^1 u_i' u_j' dx + r \int_0^1 u_i' u_j dx + c \int_0^1 u_i u_j dx \right] a_i = \int_0^1 f u_j dx - b u_j(1),$$

$$j \in S_{NE}.$$

Step 4: specifying basis functions at internal points

Let $u_i(x)$, $i = 0, \dots, n$; be linear Lagrange functions.

At the internal grid nodes $i = 1, \dots, n - 1$:

$$u_i(x) = \begin{cases} \frac{x - x_{i-1}}{x_i - x_{i-1}} & \text{if } x_{i-1} \leq x \leq x_i \\ \frac{x_{i+1} - x}{x_{i+1} - x_i} & \text{if } x_i \leq x \leq x_{i+1} \\ 0 & \text{otherwise} \end{cases}$$

Step 5: basis functions at Dirichlet boundary

Since we have nonzero Dirichlet condition $u(0) = a$, we need 'half' of the basis function at the left boundary:

$$u_0(x) = \begin{cases} 0 & \text{if } x < 0 \\ \frac{x_1 - x}{x_1} & \text{if } 0 \leq x \leq x_1 \\ 0 & \text{otherwise} \end{cases}$$

Note that $u_0(0) = 1$ and $u_i(0) = 0$ for $i = 1, \dots, n$.

Now we can also say that S_{NE} is $j = 1, \dots, n$. This means that there are n Galerkin equations. But we have $n + 1$ coefficients a_i , $i = 0, \dots, n$.

We use the Dirichlet BC $u(0) = a$ to find that $a_0 = a$:

$$u(0) = \sum_{i=0}^n a_i u_i(0) = a_0 u_0(0) = a$$

Hence, there remains n unknown a_i 's to be found and we have exactly n Galerkin equations.

Step 6: basis functions at Neumann boundary

Since right boundary $x_n = 1$ is the Neumann boundary, we need 'half' of a basis function there as well:

$$u_n(x) = \begin{cases} \frac{x-x_{n-1}}{1-x_{n-1}} & \text{if } x_{n-1} \leq x \leq 1 \\ 0 & \text{if } x > 1 \\ 0 & \text{otherwise} \end{cases}$$

We note that $u_n(1) = 1$ and $u_i(1) = 0$, $i = 0, \dots, n-1$.

Step 7: integration rule

Choose one of the quadrature rules and stick with it in all subsequent calculations, e.g., trapezoid rule:

$$\int_{x_{i-1}}^{x_i} g(x) dx \approx \frac{x_i - x_{i-1}}{2} [g(x_{i-1}) + g(x_i)]$$

It uses only the node values of $g(x)$.

Step 9: element matrices

In 1D: elements e_k , $k = 1, \dots, n$; are intervals between two neighboring nodes. For each (i, j) -entry of the matrix the integral can be split into sum of integrals over elements. For example:

$$\int_0^1 u'_i u'_j dx = \sum_{k=1}^n \int_{e_k} u'_i u'_j dx$$

Instead of focusing on a single (i, j) -entry, focus on a single element e_k . Compute all nonzero integrals that can be obtained by changing i and j for this element and collect them in the element matrix S_k :

$$S_k = \begin{bmatrix} \int_{e_k} u'_{k-1} u'_{k-1} dx & \int_{e_k} u'_{k-1} u'_k dx \\ \int_{e_k} u'_k u'_{k-1} dx & \int_{e_k} u'_k u'_k dx \end{bmatrix}$$

Do this for $k = 1, \dots, n$.

Step 10: overlap in element matrices

Notice that there is an overlap in basis functions between element matrices:

$$S_k = \begin{bmatrix} \int_{e_k} u'_{k-1} u'_{k-1} dx & \int_{e_k} u'_{k-1} u'_k dx \\ \int_{e_k} u'_k u'_{k-1} dx & \int_{e_k} \textcolor{blue}{u'_k u'_k} dx \end{bmatrix}$$
$$S_{k+1} = \begin{bmatrix} \int_{e_{k+1}} \textcolor{blue}{u'_k u'_k} dx & \int_{e_{k+1}} u'_k u'_{k+1} dx \\ \int_{e_{k+1}} u'_{k+1} u'_k dx & \int_{e_{k+1}} u'_{k+1} u'_{k+1} dx \end{bmatrix}$$

Step 11: matrix assembly

Hence, the matrix S should be assembled by inserting element matrices S_k along the main diagonal of S with an 'overlap', so that the integrals containing the same combinations of basis functions are added together in a single (i,j) -entry of S . In 1D this happens only along the main diagonal:

$$\text{(part of) } S = \begin{bmatrix} \int_{e_1} u'_0 u'_0 dx & \int_{e_1} u'_0 u'_1 dx & 0 \\ \int_{e_1} u'_1 u'_0 dx & \int_{e_1} u'_1 u'_1 dx + \int_{e_2} u'_1 u'_1 dx & \int_{e_2} u'_1 u'_2 dx \\ 0 & \int_{e_2} u'_2 u'_1 dx & \int_{e_2} u'_2 u'_2 dx + \int_{e_3} u'_2 u'_2 dx \end{bmatrix}$$

Step 12: element vectors and RHS assembly

Right-hand side vectors are also computed per element and then assembled with overlap:

$$\int_0^1 f u_j dx = \sum_{k=1}^n \int_{e_k} f u_j dx$$

$$f_k = \begin{bmatrix} \int_{e_k} f u_{k-1} dx \\ \int_{e_k} f u_k dx \end{bmatrix}$$

$$(\text{part of}) \mathbf{f} = \begin{bmatrix} \int_{e_1} f u_0 dx \\ \int_{e_1} f u_1 dx + \int_{e_2} f u_1 dx \\ \int_{e_2} f u_2 dx + \int_{e_3} f u_2 dx \end{bmatrix}$$

Step 13: moving Dirichlet condition to RHS

The first equation corresponds to $j = 0$ and we do not need it, since $a_0 = a$ is known. Hence, we can simply remove the first row of S and the first element of $\mathbf{f}$.

However, a_0 is also present in the second equation and we need to move the element

$$a_0 \int_{e_1} u'_1 u'_0 dx = a \int_{e_1} u'_1 u'_0 dx$$

to the right-hand side and add it to the corresponding element of the vector $\mathbf{f}$:

$$\begin{bmatrix} \int_{e_1} u'_1 u'_1 dx + \int_{e_2} u'_1 u'_1 dx & \int_{e_2} u'_1 u'_2 dx \\ \int_{e_2} u'_2 u'_1 dx & \int_{e_2} u'_2 u'_2 dx + \int_{e_3} u'_2 u'_2 dx \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \end{bmatrix} \dots = \begin{bmatrix} \int_{e_1} f u_1 dx + \int_{e_2} f u_1 dx - a \int_{e_1} u'_1 u'_0 dx \dots \\ \int_{e_2} f u_2 dx + \int_{e_3} f u_2 dx \end{bmatrix}$$

Further steps

- 1 Compute element matrices and assemble system matrices B and M corresponding to the terms

$$\int_0^1 u_i' u_j dx \quad \text{and} \quad \int_0^1 u_i u_j dx$$

- 2 Convert integrals into numbers before assembly!
- 3 Move Dirichlet coefficients from these matrices to the first element of the $\mathbf{f}$ vector
- 4 Add the Neumann condition to the last element of the $\mathbf{f}$ vector
- 5 Solve the linear algebraic equation:

$$[kS + rB + cM] \mathbf{a} = \mathbf{f}$$

to find the vector $\mathbf{a}$ of unknown coefficients.

Tutorial: Solving 1D Poisson's equation with FEM

Poisson's equation in 1D with non-homogeneous BC's

$$\begin{aligned} -u'' &= 0, \quad x \in (0, 1) \\ u(0) &= 3, \quad -u'(1) = 5 \end{aligned}$$

Weak formulation

Find $u \in U(\Omega) = \{u \in H^1(\Omega) \mid u(0) = 3\}$ such that

$$\int_{\Omega} (u'v') \, d\Omega = \int_{\Omega} fv \, d\Omega - \int_{\partial\Omega_N} 5v \, d\Gamma$$

for all $v \in V(\Omega) = \{v \in H^1(\Omega) \mid v(0) = 0\}$

Galerkin FEM

- Choose finite-dimensional solution (trial) space $U_h(\Omega) \subset U(\Omega)$, such that each $u_h \in U_h(\Omega)$ can be represented via the basis functions $\{u_i\}$ as

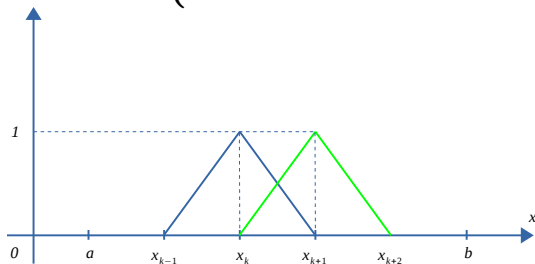
$$u_h(x) = \sum_{i=0}^n a_i u_i(x) = \sum_{i \in S_E} a_i u_i(x) + \sum_{i \in S_{NE}} a_i u_i(x)$$

where S_E is the set of essential basis functions required by the essential Dirichlet BC $u(0) = 3$, whereas, $u_i(0) = 0$, $i \in S_{NE}$

- Use for the test space $V_h(\Omega)$ a space generated by the same non-essential basis functions $\{u_i\}$, $i \in S_{NE}$

Linear Lagrange basis functions

$$u_k(x) = \begin{cases} \frac{x-x_{k-1}}{x_k-x_{k-1}} & \text{if } x_{k-1} \leq x \leq x_k \\ \frac{x_{k+1}-x}{x_{k+1}-x_k} & \text{if } x_k \leq x \leq x_{k+1} \\ 0 & \text{otherwise} \end{cases}$$



FEM for 1D Poisson with non-homogeneous BC's

On the uniform grid $h_i = h, i = 1, \dots, n$:

$$\frac{1}{h} \begin{bmatrix} 2 & -1 & 0 & \dots & 0 \\ -1 & 2 & -1 & \dots & 0 \\ \vdots & & \ddots & & \\ 0 & \dots & -1 & 2 & -1 \\ 0 & \dots & & -1 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_{n-1} \\ a_n \end{bmatrix} = h \begin{bmatrix} f(x_1) \\ f(x_2) \\ \vdots \\ f(x_{n-1}) \\ f(x_n)/2 \end{bmatrix} + \frac{1}{h} \begin{bmatrix} a \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ b \end{bmatrix}$$

FEM matrix in this case

If $[0, 1]$ is divided into 10 equal pieces, then $h = 1/10$, and, since $f = 0$, $a = 3$, and $b = 5$, we get

$$\begin{bmatrix} 20 & -10 & 0 & \dots & 0 \\ -10 & 20 & -10 & \dots & 0 \\ \vdots & & \ddots & & \\ 0 & \dots & -10 & 20 & -10 \\ 0 & \dots & & -10 & 10 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_{n-1} \\ a_n \end{bmatrix} = \begin{bmatrix} 30 \\ 0 \\ \vdots \\ 0 \\ -5 \end{bmatrix}$$

Extended FEM matrix

Sometimes the Dirichlet BC's are explicitly inserted in the system:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & \dots & 0 \\ 0 & 20 & -10 & 0 & \dots & 0 \\ 0 & -10 & 20 & -10 & \dots & 0 \\ \vdots & & & \ddots & & \\ 0 & 0 & \dots & -10 & 20 & -10 \\ 0 & 0 & \dots & & -10 & 10 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \\ a_n \end{bmatrix} = \begin{bmatrix} 3 \\ 30 \\ 0 \\ \vdots \\ 0 \\ -5 \end{bmatrix}$$

Extended FEM matrix - different source function

If $f(x) = 20$, $x \in [0, 1]$, then the system will be

$$\begin{bmatrix} 1 & 0 & 0 & 0 & \dots & 0 \\ 0 & 20 & -10 & 0 & \dots & 0 \\ 0 & -10 & 20 & -10 & \dots & 0 \\ \vdots & & & \ddots & & \\ 0 & 0 & \dots & -10 & 20 & -10 \\ 0 & 0 & \dots & & -10 & 10 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \\ a_n \end{bmatrix} = \begin{bmatrix} 3 \\ 32 \\ 2 \\ \vdots \\ 2 \\ -4 \end{bmatrix}$$

Structure of FEniCSx (dolfinx 0.9.0) program

Weak formulation used in FEniCSx and other packages

$$\int_{\Omega} (\nabla u_h \cdot \nabla v_h) d\Omega = \int_{\Omega} f v_h d\Omega - \int_{\partial\Omega} b v_h d\Gamma$$

is written as

$$a(u, v) = L(v)$$

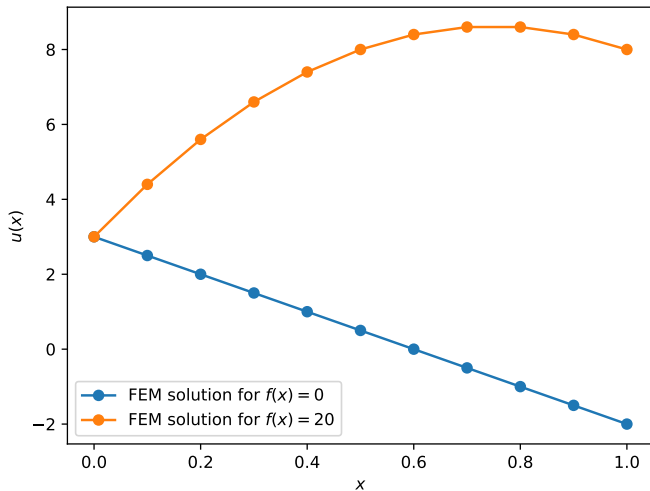
where $a(u, v)$ is the *bilinear form* and $L(v)$ is the *linear form*.

- Terms of the weak form containing both u and v are collected into $a(u, v)$, the rest goes into $L(v)$
- The boundary integral will automatically be computed over $\partial\Omega_N$, since for all test functions we have $v_h(\mathbf{x}) = 0$, $\mathbf{x} \in \partial\Omega_D$.

Minimal dolfinx code with matplotlib visualization

```
1 # FEM for 1D Poisson equation with mixed inhomogeneous BC's
2 # works in python3 and dolfinx version 0.9.0
3
4 # Neil Budka (c) 2024
5
6 import numpy as np
7 import matplotlib.pyplot as plt
8
9 import ufl
10 from ufl import dx, dv, grad, inner
11 from basix.ufl import element
12 from dolfinx import fen, mesh, __version__
13
14 from mpi4py import MPI
15 from petsc4py import PETSc
16 from petsc4py.PETSc import ScalarType
17 from dolfinx.fen.petsc import LinearProblem
18
19 dolfinx_version = '0.9.0'
20
21 if __version__ != dolfinx_version:
22     ...print('dolfinx-version-!=', __version__, ', this code requires version', dolfinx_version)
23 else:
24     ...print('dolfinx-version', dolfinx_version, 'found, solving...')
25
26 dpi=100
27
28 # input data
29 x0 = 0. # start of interval, Dirichlet BC boundary
30 Lx = 1. # length of interval
31 uDL = 1. # Dirichlet BC value
32 uNR = 1. # Neumann BC value (rest of boundary)
33 f_val = 0. # magnitude of the constant source function
34 nx = 10. # number of vertices in the mesh
35
36 # FEM solution of -u''=f, with Dirichlet and Neumann BC's, using dolfinx
37 domain = mesh.create_interval(comm=MPI.COMM_WORLD, points=(x0, x0+Lx), nx=nx,)
38 P1 = element("Lagrange", cell=domain.topology.cell_name(), degree=1)
39 V = fen.functionspace(domain, P1)
40 dofs_L = fen.locate_dofs_geometrical(V, lambda x: np.isclose(x[0], x0))
41 bc_D = fen.dirichletbc(value=ScalarType(uDL), dofs=dofs_L, V=V)
42 f = fen.Constant(domain, ScalarType(f_val))
43 u = ufl.TrialFunction(V)
44 v = ufl.TestFunction(V)
45 a = inner(grad(u), grad(v))*dx
46 L = f*v*dx - ScalarType(uNR)*v*ds
47 u = fen.Function(V)
48 bilinear_form = fen.form(a)
49 linear_form = fen.form(L)
50 problem = LinearProblem(a, L, bcs=[bc_D], petsc_options={"ksp_type": "preonly", "pc_type": "lu"})
51 u = problem.solve()
52
53 # plotting with Matplotlib
54 dm = domain.geometry.din
55 gridDof = V.tabulate_dof_coordinates()[0, dm-1]
56 uarr = u.x.array
57 plt.ion()
58 plt.figure(1, dpi=dpi)
59 plt.clf()
60 fig1, ax = plt.subplots(nrows=1, ncols=1, num=1)
61 ax.plot(gridDof, uarr, 'o', color='tab:blue', label='FEM solution')
62 ax.legend()
63 ax.set_xlabel("$x$")
64 ax.set_ylabel("$u(x)$")
65
66 print('...done')
```

FEM solutions



FEM matrix assembly in 2D on a non-uniform mesh

2D Poisson's equation with non-zero Dirichlet BC's

2D Poisson's equation with non-zero Dirichlet BC's

$$\begin{aligned} -\nabla \cdot (k \nabla u) &= f, \quad \mathbf{x} \in \Omega \subset \mathbb{R}^2 \\ u(\mathbf{x}) &= g(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega \end{aligned}$$

Weak formulation

Find $u \in U(\Omega) = \{u \in H^1(\Omega) \mid u(\mathbf{x}) = g(\mathbf{x}), \mathbf{x} \in \partial\Omega\}$ such that

$$\int_{\Omega} k (\nabla u \cdot \nabla v) \, d\Omega = \int_{\Omega} f v \, d\Omega$$

for all $v \in V(\Omega) = \{v \in H^1(\Omega) \mid v(\mathbf{x}) = 0, \mathbf{x} \in \partial\Omega\}$

Numerical solution is to be found over finite-dimensional subspaces:

- $u_h \in U_h(\Omega) = \{u_h \in H^1(\Omega) \mid u_h(\mathbf{x}) = g(\mathbf{x}), \mathbf{x} \in \partial\Omega\}, U_h \subset U$
- $v_h \in V_h(\Omega) = \{v_h \in H^1(\Omega) \mid v_h(\mathbf{x}) = 0, \mathbf{x} \in \partial\Omega\}, V_h \subset V$

$$\int_{\Omega} k (\nabla u_h \cdot \nabla v_h) \, d\Omega = \int_{\Omega} f v_h \, d\Omega$$

Galerkin method: stiffness matrix, element matrix

$$[S]_{i,j} = \int_{\Omega} k(\mathbf{x}) (\nabla u_i(\mathbf{x}) \cdot \nabla u_j(\mathbf{x})) d\Omega$$

$$[S]_{i,j}^e = \int_{\Omega_e} k(\mathbf{x}) (\nabla u_i(\mathbf{x}) \cdot \nabla u_j(\mathbf{x})) d\Omega$$

Unit (master triangle)

- Corners: $\hat{\mathbf{x}}_1 = (0, 0)$, $\hat{\mathbf{x}}_2 = (1, 0)$, and $\hat{\mathbf{x}}_3 = (0, 1)$
- Basis functions: $v_1(\hat{\mathbf{x}})$, $v_2(\hat{\mathbf{x}})$ and $v_3(\hat{\mathbf{x}})$ are defined at the corners
- Inside the triangle these basis functions have the form

$$\lambda_1(\hat{\mathbf{x}}) = 1 - \hat{x} - \hat{y},$$

$$\lambda_2(\hat{\mathbf{x}}) = \hat{x},$$

$$\lambda_3(\hat{\mathbf{x}}) = \hat{y}.$$

Iso-parametric coordinate transformation

The actual coordinates x and y inside the e -th element Ω_e with the vertices (x_1^e, y_1^e) , (x_2^e, y_2^e) , and (x_3^e, y_3^e) can be expressed through the coordinates $\hat{x}$ and $\hat{y}$ of the master element by the *iso-parametric* transformation:

$$\begin{aligned}x(\hat{x}, \hat{y}) &= x_1^e \lambda_1(\hat{x}, \hat{y}) + x_2^e \lambda_2(\hat{x}, \hat{y}) + x_3^e \lambda_3(\hat{x}, \hat{y}), \\y(\hat{x}, \hat{y}) &= y_1^e \lambda_1(\hat{x}, \hat{y}) + y_2^e \lambda_2(\hat{x}, \hat{y}) + y_3^e \lambda_3(\hat{x}, \hat{y}).\end{aligned}$$

Basis functions inside the Ω_e element

The basis functions defined at the three vertices of the element Ω_e , inside this element, have the form $\lambda_1^e(\mathbf{x}) = \lambda_1(\hat{\mathbf{x}}(\mathbf{x}))$, $\lambda_2^e(\mathbf{x}) = \lambda_2(\hat{\mathbf{x}}(\mathbf{x}))$, and $\lambda_3^e(\mathbf{x}) = \lambda_3(\hat{\mathbf{x}}(\mathbf{x}))$, that can be obtained by the coordinate transformation.

Derivatives of basis functions inside Ω_e

To evaluate the element matrix, we need the following expression:

$$\begin{aligned}\nabla u_i \cdot \nabla u_j|_{\mathbf{x} \in \Omega_e} &= \nabla \lambda_n^e \cdot \nabla \lambda_m^e = \frac{\partial \lambda_n^e(\mathbf{x})}{\partial x} \frac{\partial \lambda_m^e(\mathbf{x})}{\partial x} + \frac{\partial \lambda_n^e(\mathbf{x})}{\partial y} \frac{\partial \lambda_m^e(\mathbf{x})}{\partial y} \\ &= \frac{\partial \lambda_n(\hat{\mathbf{x}}(\mathbf{x}))}{\partial x} \frac{\partial \lambda_m(\hat{\mathbf{x}}(\mathbf{x}))}{\partial x} + \frac{\partial \lambda_n(\hat{\mathbf{x}}(\mathbf{x}))}{\partial y} \frac{\partial \lambda_m(\hat{\mathbf{x}}(\mathbf{x}))}{\partial y}, \quad n, m = 1, 2, 3.\end{aligned}$$

Transformation of partial derivatives

The partial derivatives are transformed as:

$$\begin{aligned}\frac{\partial \lambda_n}{\partial x} &= \frac{\partial \lambda_n}{\partial \hat{x}} \frac{\partial \hat{x}}{\partial x} + \frac{\partial \lambda_n}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial x}, \\ \frac{\partial \lambda_n}{\partial y} &= \frac{\partial \lambda_n}{\partial \hat{x}} \frac{\partial \hat{x}}{\partial y} + \frac{\partial \lambda_n}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial y},\end{aligned}$$

$$\nabla_{\mathbf{x}} \lambda_n^e = J_e \nabla_{\hat{\mathbf{x}}} \lambda_n,$$

$$J_e = \begin{bmatrix} \frac{\partial \hat{x}}{\partial x} & \frac{\partial \hat{y}}{\partial x} \\ \frac{\partial \hat{x}}{\partial y} & \frac{\partial \hat{y}}{\partial y} \end{bmatrix} = \begin{bmatrix} \frac{\partial x}{\partial \hat{x}} & \frac{\partial y}{\partial \hat{x}} \\ \frac{\partial x}{\partial \hat{y}} & \frac{\partial y}{\partial \hat{y}} \end{bmatrix}^{-1}$$

This is the Jacobian matrix for the element Ω_e

Element Jacobian matrix

$$\begin{aligned} J_e &= \begin{bmatrix} -x_1^e + x_2^e & -y_1^e + y_2^e \\ -x_1^e + x_3^e & -y_1^e + y_3^e \end{bmatrix}^{-1} \\ &= \frac{1}{(x_3^e - x_1^e)(y_2^e - y_1^e) - (x_2^e - x_1^e)(y_3^e - y_1^e)} \begin{bmatrix} y_3^e - y_1^e & y_1^e - y_2^e \\ x_1^e - x_3^e & x_2^e - x_1^e \end{bmatrix} \\ &= \begin{bmatrix} J_{11}^e & J_{12}^e \\ J_{21}^e & J_{22}^e \end{bmatrix}. \end{aligned}$$

Explicit expressions for partial derivatives of basis functions

$$\frac{\partial \lambda_1}{\partial x} = J_{11}^e \frac{\partial \lambda_1}{\partial \hat{x}} + J_{12}^e \frac{\partial \lambda_1}{\partial \hat{y}} = -J_{11}^e - J_{12}^e,$$

$$\frac{\partial \lambda_2}{\partial x} = J_{11}^e \frac{\partial \lambda_2}{\partial \hat{x}} + J_{12}^e \frac{\partial \lambda_2}{\partial \hat{y}} = J_{11}^e,$$

$$\frac{\partial \lambda_3}{\partial x} = J_{11}^e \frac{\partial \lambda_3}{\partial \hat{x}} + J_{12}^e \frac{\partial \lambda_3}{\partial \hat{y}} = J_{12}^e,$$

$$\frac{\partial \lambda_1}{\partial y} = J_{21}^e \frac{\partial \lambda_1}{\partial \hat{x}} + J_{22}^e \frac{\partial \lambda_1}{\partial \hat{y}} = -J_{21}^e - J_{22}^e,$$

$$\frac{\partial \lambda_2}{\partial y} = J_{21}^e \frac{\partial \lambda_2}{\partial \hat{x}} + J_{22}^e \frac{\partial \lambda_2}{\partial \hat{y}} = J_{21}^e,$$

$$\frac{\partial \lambda_3}{\partial y} = J_{21}^e \frac{\partial \lambda_3}{\partial \hat{x}} + J_{22}^e \frac{\partial \lambda_3}{\partial \hat{y}} = J_{22}^e,$$

Compact form

$$\lambda_x^e = \begin{bmatrix} -J_{11}^e - J_{12}^e \\ J_{11}^e \\ J_{12}^e \end{bmatrix}, \quad \lambda_y^e = \begin{bmatrix} -J_{21}^e - J_{22}^e \\ J_{21}^e \\ J_{22}^e \end{bmatrix}.$$

$$M_e = \lambda_x^e (\lambda_x^e)^T + \lambda_y^e (\lambda_y^e)^T.$$

Element matrix

$$S_e = M_e \int_{\Omega_e} k(\mathbf{x}) d\Omega = M_e A_e \bar{k}_e,$$

$$A_e = \frac{1}{2}(\mathbf{x}_2^e - \mathbf{x}_1^e) \cdot (\mathbf{x}_3^e - \mathbf{x}_1^e),$$

$$\bar{k}_e = \frac{1}{A_e} \int_{\Omega_e} k(\mathbf{x}) d\Omega \approx \frac{1}{3}(k(\mathbf{x}_1^e) + k(\mathbf{x}_2^e) + k(\mathbf{x}_3^e))$$